ARQUITECTURA DE COMPUTADORES

2º curso del Grado en Ingeniería Informática – Ingeniería del Software
Departamento de Arquitectura y Tecnología de Computadores. Universidad de Sevilla.

Boletín para las sesiones de laboratorio

MEMORIA CACHÉ

Técnicas de optimización de software

OBJETIVOS

El objetivo principal de esta práctica es analizar cómo el rendimiento de un programa se ve afectado por la secuencia de accesos a la memoria que realiza. En concreto, se estudiará cómo optimizar el software (el código fuente de alto nivel) para minimizar el número de fallos en caché y, de esta forma, reducir significativamente el tiempo de ejecución. Se presentarán las siguientes estrategias o técnicas de optimización de software:

- Intercambio de bucles
- Procesamiento por bloques

Además, la realización de esta práctica permitirá consolidar algunos de los conceptos relacionados con la memoria caché que se han estudiado en las clases de teoría.

La práctica presenta varios tipos de ejercicios:

- Ejercicios teóricos que el alumno debe realizar en papel y que pueden requerir el análisis de pequeños programas escritos en C o en el juego de instrucciones del RISC-V.
- Ejercicios prácticos que implican el estudio y ejecución de pequeños programas escritos en C. Dependiendo del ejercicio, estos programas serán ejecutados en el simulador RARS o en el procesador del PC.
- Ejercicios prácticos en los que el alumno debe aplicar las técnicas de optimización a pequeños programas escritos en C

Para la consecución de estos objetivos, se plantean ejercicios a resolver por el alumno mediante la ejecución de código RISC-V con el simulador RARS y en el procesador nativo basado en la arquitectura x86 de la máquina en la que se realiza este boletín.

REQUISITOS

Los requisitos de la práctica son los siguientes:

- Haber estudiado los conceptos de teoría asociados a las memorias cachés (Tema 4: «Sistemas de Memoria»)
- Realizar una lectura previa de este boletín antes de asistir a clase.
- Para realizar la práctica en casa, se necesita tener instalado:
    - Una máquina con el sistema operativo Ubuntu 24.04 (o posterior) o Windows con WSL (Windows Subsystem for Linux).
    - El simulador RARS.
    - El compilador gcc y el *toolchain* de RISC-V para gcc (véase la Práctica 1, Sección 4.1).

HERRAMIENTAS

En este apartado se van a mostrar las herramientas que el alumno va a emplear para realizar los ejercicios del boletín.

El simulador de caché de RARS

Bajo la opción del menú *Tools*, RARS proporciona algunas de las herramientas que pueden ser útiles para comprender mejor el funcionamiento de algunos de los elementos que constituyen la arquitectura de un ordenador. Entre estas herramientas se encuentra un simulador de caches, al que se accede a través de *Tools ->Data Cache Simulator.*

La ventana del simulador de cachés de RARS (ver Figura 1) se encuentra dividida en dos partes. La parte superior, *Cache Organization*, determina el tipo de caché, mientras que la parte inferior, *Cache Performance,* muestra el rendimiento de la caché durante la ejecución de un determinado programa.

El simulador es capaz de simular los tres tipos de cachés estudiadas en teoría. El tipo de caché se puede seleccionar mediante la opción *Placement Policy*, que admite los siguientes opciones:

- *Direct Mapping*: Caché de correspondencia directa o mapeado directo. Es la opción configurada por defecto.
- *N-way Set Associative*. Caché asociativa por conjuntos de N-vías.
- *Fully Associative*: Caché totalmente asociativa.

En todos los casos se pueden simular tanto bloques (o líneas) de una palabra como bloques de varias palabras, siendo el tamaño de la palabra de 32 bits. El número de palabras de un bloque se puede fijar mediante la opción *Caché Block Size (words).*

En el caso de las cachés asociativas, el algoritmo de reemplazo se selecciona mediante la opción *Block Replacement Policy.* El simulador nos proporciona dos tipos de algoritmos de reemplazo: *LRU* (opción por defecto) y *RANDOM*.

Por otro lado, el número de vías, en el caso de una caché asociativa por conjuntos de N-vías, se puede fijar mediante la opción *Set Size (blocks).*

El tamaño de la caché *Cache Size (bytes)* viene determinado por el número de líneas de ésta (*Number of Blocks)* y el tamaño de cada línea *(Caché Block Size (words)). En función del tipo de caché, el número de bloques determina:*

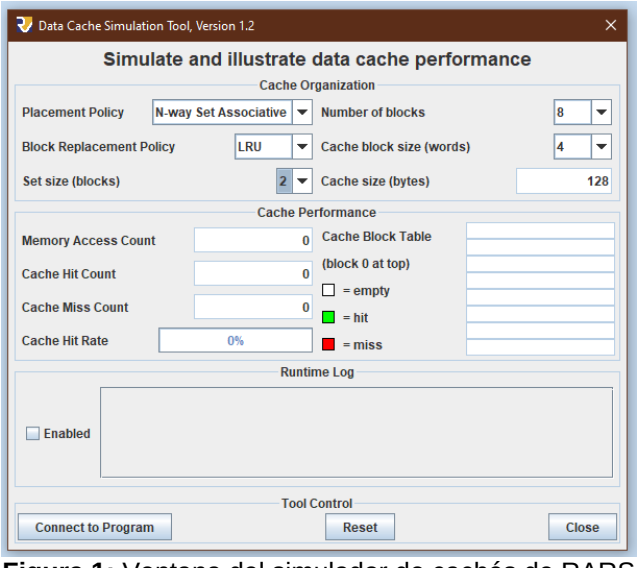

Figura 1: Ventana del simulador de cachés de RARS

La parte inferior de la ventana del simulador de cachés muestra el rendimiento de la caché durante la ejecución de un determinado programa. En esta zona se puede consultar el número de fallos (*Cache Miss Count*) o la frecuencia de aciertos (*Cache Hit Rate*), además de otros datos. Para que se muestre esta información, antes de la ejecución del programa, tendremos que "conectar" el simulador de cachés con el programa que se está simulando, pulsando el botón *Connect to Program.*

Adicionalmente, es recomendable habilitar el volcado de las trazas (*Runtime Log*) activando la opción *Enabled*. De este modo, al ejecutar un programa paso a paso, cuando se ejecute una instrucción que lleve a cabo un acceso a memoria, puede observarse si se ha producido acierto o fallo.

Un aspecto importante de esta ventana es que si se ha finalizado una ejecución y se desea repetir, es necesario reiniciar los datos presionando el botón *RESET.*

Por último, hay dos aspectos del simulador que hay que tener en cuenta. El primero es que el simulador sólo simula una caché de datos. Por tanto, la ejecución de las propias instrucciones no produce ni aciertos ni fallos de caché. El segundo es que la caché utiliza como política de escritura la «Escritura Directa» (*Write Through*), en caso de acierto, y la «Ubicación en escritura» (*Write Allocate*), en caso de fallo.

Compilación del código C

Compilación cruzada para RISC-V

Al igual que se hizo en la práctica 1, en algunos ejercicios del boletín se ejecutarán en el simulador RARS pequeños programas escritos en C. Esto permitirá analizar cómo afecta las técnicas de optimización que estudiaremos al número de fallos de la caché. Por lo tanto, el código C deberá ser compilado al juego de instrucciones del procesador RISC-V que simula RARS. Para realizar esta *compilación cruzada*, se proporcionan dos aplicaciones que están disponibles en Enseñanza Virtual junto con este boletín:

- `compilacion-cruzada.sh`
- `limpia-asm.py`

El script *compilacion-cruzada.sh* ejecutará el compilador *gcc* para generar el código ensamblador del RISC-V y, posteriormente, ejecutará automáticamente el script *limpia-asm.py* para adaptar dicho código a la sintaxis de RARS.

Sintaxis del script de compilación:

```
compilacion-cruzada.sh fichero_fuente
```

Pasos para compilar:

En primer lugar, debe copiar ambos scripts a la carpeta en la que vaya a guardar las soluciones de los ejercicios.

Puede compilar su código de dos formas. Se recomienda la primera opción, ya que no requiere modificar los permisos del fichero:

- **Opción A (Recomendada).** Invoque el script directamente con el intérprete *bash*. Por ejemplo, para compilar el código contenido en un archivo llamado “ejemplo.c”, debe ejecutar el siguiente comando:

  ```
  bash compilacion-cruzada.sh ejemplo.c
  ```

- **Opción B.** Si prefiere ejecutarlo directamente como un programa (sin usar *bash*), primero debe asignarle permisos de ejecución al script:

  ```
  chmod +x compilacion-cruzada.sh
  ```

  A partir de este momento, puede ejecutar la compilación como si fuera un programa:

  ```
  ./compilacion.sh ejemplo.c
  ```

  El script invocará al compilador riscv64-linux-gnu-gcc con los parámetros adecuados (-march=rv32im -mabi=ilp32) para generar código compatible con el simulador RARS.

Si no ha habido errores de compilación, el script generará un fichero con código ensamblador que se puede cargar directamente en el simulador RARS. Dicho fichero tendrá el prefijo **rars**_ y la extensión **.s**; por ejemplo:

```
rars_ejemplo.s
```

Compilación nativa para x64

Mientras que en algunos ejercicios estudiaremos el comportamiento de la caché con el simulador RARS, en otros ejecutaremos el código C en el procesador del propio PC, con arquitectura x64 de Intel. Esto nos permitirá medir tiempos de CPU y comprobar cómo afecta la reducción de fallos de caché al rendimiento del programa.

Para compilar el código C y generar un ejecutable para el procesador del PC (denominaremos a este proceso *compilación nativa*), utilizaremos directamente el compilador *gcc*.

Comando de compilación:

```
gcc -OX -o fichero_ejecutable fichero_fuente
```

donde X es un valor entero de 0 a 3 que indica el nivel de optimización que empleará el compilador al generar el archivo ejecutable. Por ejemplo, el siguiente comando:

```
gcc -O2 -o mi_programa ejemplo.c
```

Compila el código C del archivo fuente "*ejemplo.c*" utilizando el nivel 2 de optimización y genera un archivo ejecutable llamado "*mi_programa*". Dicho programa se podrá ejecutar el terminal escribiendo

```
./mi_programa
```

TÉCNICAS DE OPTIMIZACIÓN DE SOFTWARE

Las técnicas que se presentan a continuación tienen como objetivo reducir la tasa de fallos de las memorias cachés en la ejecución de un código. Con ello se pretende mejorar el tiempo medio de acceso a memoria caché:

$$t_{mam} = t_{acierto} + f_{fallos} \times p_{fallo}$$

donde $t_{acierto}$ es el tiempo de acierto; f_{fallos}, la frecuencia de fallos y p_{fallo}, la penalización por fallo. De estos tres factores que afectan al tiempo medio de acceso a memoria, el software puede influir positiva o negativamente en la frecuencia de fallos. Estudiaremos técnicas de optimización diseñadas para mejorar la **localidad espacial y temporal** de los accesos a memoria que realiza el código, lo que permite reducir los fallos de caché.

En general, los mecanismos de mejora son aplicables en caso de tener un conjunto grande y repetido de accesos consecutivos a memoria en el programa, que es donde se puede hacer uso y mejora de los principios de localidad anteriormente mencionados. Un ejemplo de ello serían los **bucles (localidad temporal) y las matrices (localidad espacial)**.

En las siguientes secciones se presentarán cada una de las técnicas junto con los ejercicios relacionados.

TÉCNICA 1: Intercambio de bucles

Mediante esta técnica se pretende cambiar la manera en la que un bucle trata los elementos de una matriz para que éste haga mejor uso de la localidad espacial de las variables en memoria. En lenguajes como C, una matriz de dos dimensiones está organizada en memoria RAM por filas (es decir, los elementos de la misma fila son adyacentes en memoria).

Un bucle que procese los elementos de una matriz por columnas (es decir, primero procesa todos los elementos de la misma columna, para toda matriz), tendrá un aprovechamiento peor de la localidad espacial que otro que

trabaja por filas (procesando todos los elementos de la misma fila). En la siguiente figura se muestra cómo se suceden los accesos a una matriz de 3x3 elementos de tamaño palabra, cuando se accede por filas (imagen superior) y por columnas (imagen inferior):

Acceso a una fila de la matriz:

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$

Su implementación en memoria:

Dir	Dato
0x0000	$A_{0,0}$
0x0004	$A_{0,1}$
0x0008	$A_{0,2}$
0x000C	$A_{1,0}$
0x0010	$A_{1,1}$
0X0014	$A_{1,2}$
0x001C	$A_{2,0}$
0x0020	$A_{2,1}$
0X0024	$A_{2,2}$

Acceso a una columna de la matriz:

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$

Su implementación en memoria:

Dir	Dato
0x0000	$A_{0,0}$
0x0004	$A_{0,1}$
0x0008	$A_{0,2}$
0x000C	$A_{1,0}$
0x0010	$A_{1,1}$
0X0014	$A_{1,2}$
0x001C	$A_{2,0}$
0x0020	$A_{2,1}$
0X0024	$A_{2,2}$

EJERCICIO 1.1: Estudio teórico.

El fichero “RISCV_intercambio_ej1p1.c” contiene un programa que inicializa una matriz con un determinado valor mediante dos bucles anidados. Estudie el código del programa. Puede observar que las variables locales se han declarado utilizando el modificador ***register***:

```
register int i, j;
```

En la declaración anterior, el modificador *register* indica al compilador que use registros para contener los valores de las variables; como consecuencia, dichas variables no se guardarán en memoria. Esto es importante en este ejercicio (y en los siguientes) debido a que el objetivo es estudiar como reducir los fallos de caché que se producen al acceder a la matriz. Como regla general en todos los ejercicios, todas las variables locales de la función *main* las declararemos con el modificador *register*.

Estudie el código del programa y realice los siguientes apartados suponiendo un sistema con las siguientes características:

- Caché totalmente asociativa de 8 líneas, con bloques de 4 palabras de 32 bits.
- La matriz A se encuentra en la dirección 0x10010000 de memoria.

a) Indique los campos en los que la caché divide las direcciones físicas (para cada campo especifique el nombre y el tamaño en bits).

b) Rellene la siguiente tabla para los 9 primeros accesos a memoria:

Dirección (hex)	Etiqueta (hex)	Desplazamiento (hex)	Fallo/acierto

c) ¿De qué tipo son los fallos que aparecen en la tabla? ¿Cabe la matriz en la caché? ¿Qué tipo de fallos se producirán cuando comience el procesamiento de la segunda columna? ¿Cuál será la frecuencia de fallo? ¿Cuál es el número total de accesos?

Tipo de fallo	Frec. Fallos	Nº accesos

EJERCICIO 1.2: Estudio experimental con RARS

Compruebe mediante el simulador RARS que los accesos que ha calculado son correctos realizando los siguientes pasos:

- Compile el código para el RISC-V (compilación cruzada) y cárguelo en el simulador RARS. Abra la herramienta "***Data caché simulator***", configure una caché con las características del ejercicio anterior y conecte la herramienta al programa. A continuación, en el código ensamblador, busque la escritura en memoria (instrucción sw) que se realiza dentro del bucle. Ejecute paso a paso el programa hasta la instrucción previa. No olvide activar el ***"runtime log"*** en dicha herramienta.
- A continuación, ejecute la instrucción *sw* anteriormente citada; verá cómo se actualizan las estadísticas en dicha ventana y se describe qué es lo que ha pasado en la ventana de *LOG*. De esa forma, comprobará el primer acceso. Si establece un **punto de interrrupción** en dicha instrucción, podrá verificar uno por uno el resto de accesos, dándole al botón de ejecutar (F5) y haciendo que el programa pare en cada una de las instrucciones *sw* que le interesan.
- Ejecute por completo el programa y compruebe que la tasa de fallo y el número de accesos totales son los que calculó anteriormente. ¿Porqué hay más accesos a memoria de los que ha calculado anteriormente?

Razonamiento:

EJERCICIO 1.3: Optimización del código.

Copie el programa del Ejercicio 1.1 en un nuevo archivo y optimice el código con la técnica de intercambio de bucles para que realice un recorrido por filas de la matriz en vez de por columnas. A continuación, realice y los siguientes apartados:

a) Rellene la tabla para los primeros 9 accesos a memoria que realiza el programa:

Dirección (hex)	Etiqueta (hex)	Desplazamiento (hex)	Fallo/acierto

b) ¿Cuál es la nueva tasa de fallo?¿Porqué mejora? ¿Qué tipo de fallo se elimina? ¿Qué tipo de localidad se está aprovechando?

Motivo de la mejora	Frec. De Fallos	Tipo de fallos eliminados	Localidad

c) Por último, compruebe de nuevo sus cálculos mediante simulación cruzada en RARS.

EJERCICIO 1.4: Análisis de prestaciones en el procesador nativo.

En este ejercicio, estudiaremos cómo afectan al rendimiento del programa la optimización realizada en los ejercicios anteriores. El fichero "x64_intercambio_ej1p4.c" contiene una versión del programa del Ejercicio 1.1, que procesa matrices de mayor tamaño. Estudie el código y realice los siguientes apartados:

a) Compile el código para el procesador del PC (compilación nativa) utilizando el nivel 0 de optimización, ejecute el programa y obtenga el tiempo de ejecución en segundos con el comando *time*, tal como se muestra a continuación:

```
/usr/bin/time -f "%e" ./archivo_ejecutable
```

Anote el tiempo de reloj que muestra dicho comando.

b) Copie el programa en un nuevo archivo, optimice el código para que realice el recorrido de la matriz por filas y calcule el tiempo de ejecución de la versión optimizada. Calcule el factor de aceleración obtenido con la optimización del código respecto a la versión sin optimizar.

c) Repita los apartados anteriores utilizando el nivel 3 de optimización. ¿Qué aceleración se obtiene? ¿Se podría decir que la optimización del compilador ha conseguido reducir los fallos de caché de forma similar a la técnica software que hemos empleado?

EJERCICIO 1.5

En el ejercicio anterior comprobamos que las dos versiones del programa (recorrido por filas y por columnas) obtenían un rendimiento similar cuando se compilaba con el nivel 3 de optimización del compilador. Esto puede darnos la falsa impresión de que estas técnicas de optimización de caché son innecesarias debido a las optimizaciones del compilador. Es decir, que no tienen ningún impacto en el rendimiento cuando se usa este nivel de optimización. Sin embargo, en este ejercicio comprobaremos que esta conclusión no es correcta. Cuando el procesamiento de la matriz es más complejo, las técnicas de optimización de caché tienen un impacto positivo incluso utilizando el nivel 3 de optimización.

El fichero "x64-max-columnas-ej1p5.c" contiene un programa que calcula el máximo de cada columna de una matriz. Estudie el código y realice los siguientes apartados:

a) Realice la compilación nativa del código utilizando el nivel 3 de optimización. Ejecute el programa y obtenga el tiempo de ejecución. En este caso, no es necesario utilizar el comando *time* porque el propio programa muestra en el terminal el tiempo empleado en el procesamiento de la matriz. El tiempo mostrado no incluye la inicialización de la matriz porque solo estudiaremos el impacto de la técnica en el cálculo del máximo.

b) Copie el programa en un nuevo archivo y aplique la técnica de intercambio de bucles al bloque de código que calcula el máximo de cada columna (no aplique la técnica a la inicialización de la matriz). En este caso, la aplicación de la técnica no es tan simple como en el Ejercicio 1.4, ya que requiere realizar algunas modificaciones adicionales además de intercambiar los bucles.

c) Compile el nuevo programa con el nivel 3 de optimización, ejecútelo y calcule la aceleración que se obtiene.

TÉCNICA 2: Procesamiento por bloques

Si un programa procesa un conjunto de datos grande que no cabe en la caché, pueden producirse fallos de capacidad cuando dicho procesamiento implica un acceso no secuencial a la memoria. En términos generales, la técnica de procesamiento por bloques consiste en dividir el conjunto de datos en subconjuntos más pequeños que quepan en la caché (que denominaremos *bloques*, pero no debe confundirse este término con los bloques de memoria). El objetivo es procesar completamente cada bloque de datos antes de pasar al siguiente bloque para evitar, de forma que el bloque procesado no vuelva a utilizarse.

Los siguientes ejercicios nos permitirán estudiar con más detalle esta técnica.

EJERCICIO 2.1: Planteamiento del problema: trasposición de matrices.

El fichero “RISCV_bloques_noopt_ej2p1.c” contiene un programa que procesa dos matrices llamadas A y B. El programa copia en la matriz B la traspuesta de la matriz A en la matriz B. Estudie el código del programa y realice los siguientes apartados suponiendo un sistema con las siguientes características:

- Caché totalmente asociativa de 8 líneas, con bloques de 4 palabras de 32 bits. Se trata de la misma caché que en el Ejercicio 1.1.
- La matriz A se encuentra en la dirección 0x10010000 de memoria.
- La matriz B se encuentra en la dirección 0x10010100 de memoria. Las direcciones de memoria de ambas matrices no son realmente necesarias para realizar este ejercicio, pero se proporcionan por si el alumno las necesitara.

a) En el diagrama siguiente se muestran los bloques en memoria de las matrices A y B. Cada bloque contiene 4 valores de tipo *int* (4 palabras); es decir, cada bloque contiene 4 elementos de una fila. Los bloques están numerados en el mismo orden en que se encuentran en memoria.

Bloques de la matriz A

A0	A1
A2	A3
A4	A5
A6	A7
A8	A9
A10	A11
A12	A13
A14	A15

Bloques de la matriz B

B0	B1
B2	B3
B4	B5
B6	B7
B8	B9
B10	B11
B12	B13
B14	B15

El siguiente diagrama, que representa las líneas de la caché de datos. Realice una traza de las 8 primeras iteraciones del bucle interno y rellene a lápiz el contenido de la caché (indicando solamente el bloque que hay en cada línea). ¿Cuántos fallos se producen y de qué tipo son?

Líneas de caché

0	
1	
2	
3	
4	
5	
6	
7	

b) Partiendo del estado de la caché en el apartado anterior, realice una traza de las 8 siguientes iteraciones del bucle interno y actualice el contenido de la caché (a lápiz). ¿Cuántos fallos se producen? ¿Qué tipo de fallos son?

c) ¿Cuántos fallos se producirán al ejecutar el programa completo? ¿Cuál será la frecuencia de fallos?

d) Compile y simule el código en RARS para comprobar que los cálculos del apartado anterior son correctos.

EJERCICIO 2.2: Aplicación de la técnica de optimización de procesamiento por bloques.

Realice los siguientes apartados:

a) ¿La técnica de intercambio de bucles puede reducir el número de fallos de caché en el programa del Ejercicio 2.1?¿Por qué?

b) En el fichero “RISCV_bloques_optimizado_ej2p2.c” se muestra cómo aplicar la técnica de procesamiento por bloques a la trasposición de matrices. Estudie el código del programa. Si no consigue entender el código, pregunte a su profesor de prácticas.
c) Repita el Ejercicio 2.1 para el nuevo código.

EJERCICIO 2.3: Análisis de prestaciones con el simulador.

El objetivo de las técnicas de optimización estudiadas es obtener una mejora del rendimiento basada en la reducción de fallos de caché (es decir, en reducir el tiempo medio de acceso a memoria). Algunas de estas técnicas requieren cambios en el código fuente que implican la ejecución de un mayor número de instrucciones. Como consecuencia, el incremento del tiempo de CPU debido a las instrucciones nuevas podría contrarrestar la mejora obtenida. En este ejercicio estudiaremos este efecto utilizando el simulador RARS.

Para obtener el número de instrucciones ejecutadas podemos utilizar la herramienta de estadísticas (véase la Figura 2) que se encuentra en el menú de herramientas (*Tools -> Instruction Statistics*).

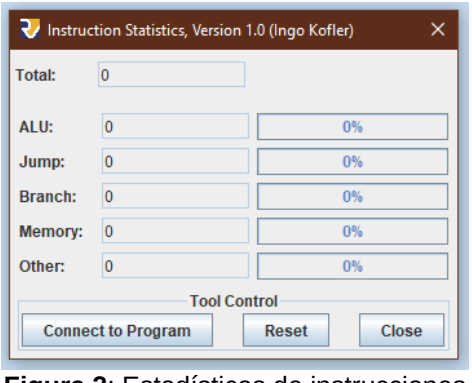

Figura 2: Estadísticas de instrucciones

Esta herramienta funciona de manera similar a la de caché: para que contabilice instrucciones, ha de conectarse al programa, de manera que puede conectarla y desconectarla cuando le convenga. Y puede resetear el conteo con el botón “Reset”.

Realice los siguientes apartados suponiendo lo siguiente:

- La caché es totalmente asociativa con 16 bloques de 4 palabras (el tamaño total de la caché será de 256 bytes). Política de reemplazo LRU.
- Por defecto, el CPI de todas las instrucciones es 5
- Las instrucciones que acceden a memoria y que provocan un fallo de caché tienen una penalización de 10 ciclos.

a) Realice los siguientes cambios en los programas de los Ejercicios 2.1 y 2.2:

- El valor de **N** debe ser **16**
- En "RISCV_bloques_optimizado_ej2p2.c", el valor de **BSIZE** debe ser **8**

Tras realizar los cambios, simule ambos programas con RARS y rellene la siguiente tabla:

	Código no optimizado (Ejercicio 2.1)	Código optimizado (Ejercicio 2.2)
Nº de instrucciones de memoria ejecutadas		
Nº total de instrucciones ejecutadas		
Frecuencia de fallos en caché		

b) Usando los datos de la tabla anterior, calcule el tiempo de ejecución de ambos programas y el factor de aceleración obtenido con la versión optimizada. Rellene la siguiente tabla:

	Código no optimizado (Ejercicio 2.1)	Código optimizado (Ejercicio 2.2)
Tiempo de ejecución		
Aceleración		

c) Cambie el valor de **BSIZE** a **4** y vuelva a simular el programa "RISCV_bloques_optimizado_ej2p2.c". Repita los apartados (a) y (b) con los nuevos datos obtenidos y conteste a las siguientes preguntas: ¿Sigue siendo más rápido el código optimizado? ¿A qué se debe ese cambio en el rendimiento?

EJERCICIO 2.5: Análisis de prestaciones en el procesador nativo.

En este ejercicio, estudiaremos cómo afectan al rendimiento del programa la optimización realizada en los ejercicios anteriores. Los ficheros "x64_bloques_noopt_ej2p5.c" y "x64_bloques_optimizado_ej2p5.c" contienen versiones de los programas del Ejercicio 2.1, que procesan matrices de mayor tamaño. Estudie el código de ambos ficheros y realice los siguientes apartados:

a) Compile ambos programas para el procesador nativo utilizando el nivel 0 de optimización, ejecútelos y calcule la aceleración.

b) Repita el apartado (a) utilizando el nivel de optimización 3. ¿Consigue la optimización del compilador aumentar las prestaciones de ambos programas?

EJERCICIO 2.6

El archivo "x64_ejercicio2p6.c" contiene un programa que procesa 3 matrices denominadas A, B y C, y que calcula:

$$C = A + B^T$$

a) Compile y ejecute el programa utilizando el nivel de optimización que desee.

b) Copie el programa en un nuevo archivo y optimice los accesos a la caché utilizando la técnica de procesamiento por bloques. Intente aplicar la técnica sin consultar el código de los ejercicios anteriores. Calcule la aceleración que se obtiene.